

bok25

基
礎
班

、**考研数据結構**



二叉排序树

(二叉搜索树)



二叉排序树

(二叉搜索树)

- 性质：
- 若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值。
 - 若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值。
 - 它的左右子树也分别为二叉排序树

中序遍历一棵二叉树时可以得到一个结点值递增的有序序列



二叉排序树

结构体

(二叉搜索树)

```
// 定义一个元素类型的结构体，用于存储二叉搜索树的节点数据
typedef struct
{
    KeyType key;           // 键类型，用于比较节点
    InfoType otherinfo;    // 附加信息类型，用于存储除键之外的其他信息
}ElemType;

// 定义一个二叉搜索树节点的结构体
typedef struct BSTNode
{
    ElemType data;         // 节点数据，包含键和其他信息
    struct BSTNode *lchild, *rchild; // 指向左子节点和右子节点的指针
}BSTNode, *BSTree; // BSTNode为节点类型，
//BSTree为指向BSTNode的指针类型，代表整个二叉搜索树
```




二叉排序树

查找规则

(二叉搜索树)

- 若二叉排序树为空，则查找失败，返回空指针
- 若二叉排序树非空，将给定值key与根节点的关键字T->data.key进行比较
 - 若key等于T->data.key，则查找成功，返回根节点地址
 - 若key小于T->data.key，则递归查找左子树
 - 若key大于T->data.key，则递归查找右子树

二叉排序树

查找规则

(二叉搜索树)

- 若二叉排序树为空，则查找失败，返回空指针
- 若二叉排序树非空，将给定值key与根节点的
关键字T->data.key进行比较

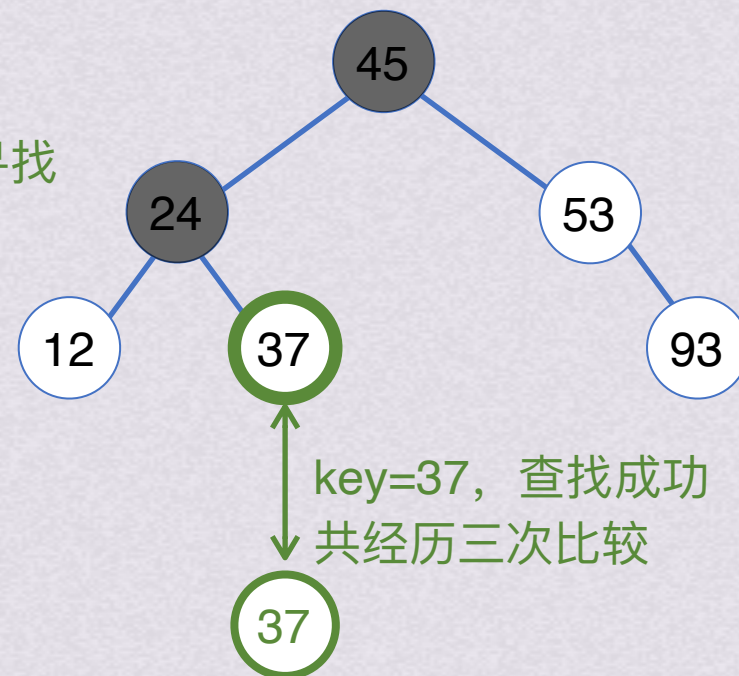
• 若key等于T->data.key，则查找成功，返回根节点地址

• 若key小于T->data.key，则递归查找左子树

• 若key大于T->data.key，则递归查找右子树

37<45，左子树中继续寻找

37>24，右子树中寻找



二叉排序树

例：在该二叉排序树中查找key=37

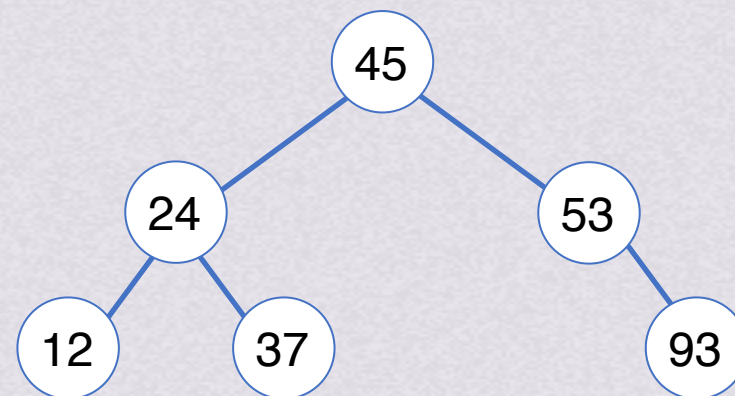


二叉排序树

查找规则

(二叉搜索树)

- 若二叉排序树为空，则查找失败，返回空指针
- 若二叉排序树非空，将给定值key与根节点的关键字T->data.key进行比较
 - 若key等于T->data.key，则查找成功，返回根节点地址
 - 若key小于T->data.key，则递归查找左子树
 - 若key大于T->data.key，则递归查找右子树



二叉排序树

例2：在该二叉排序树中查找key=54



二叉排序树

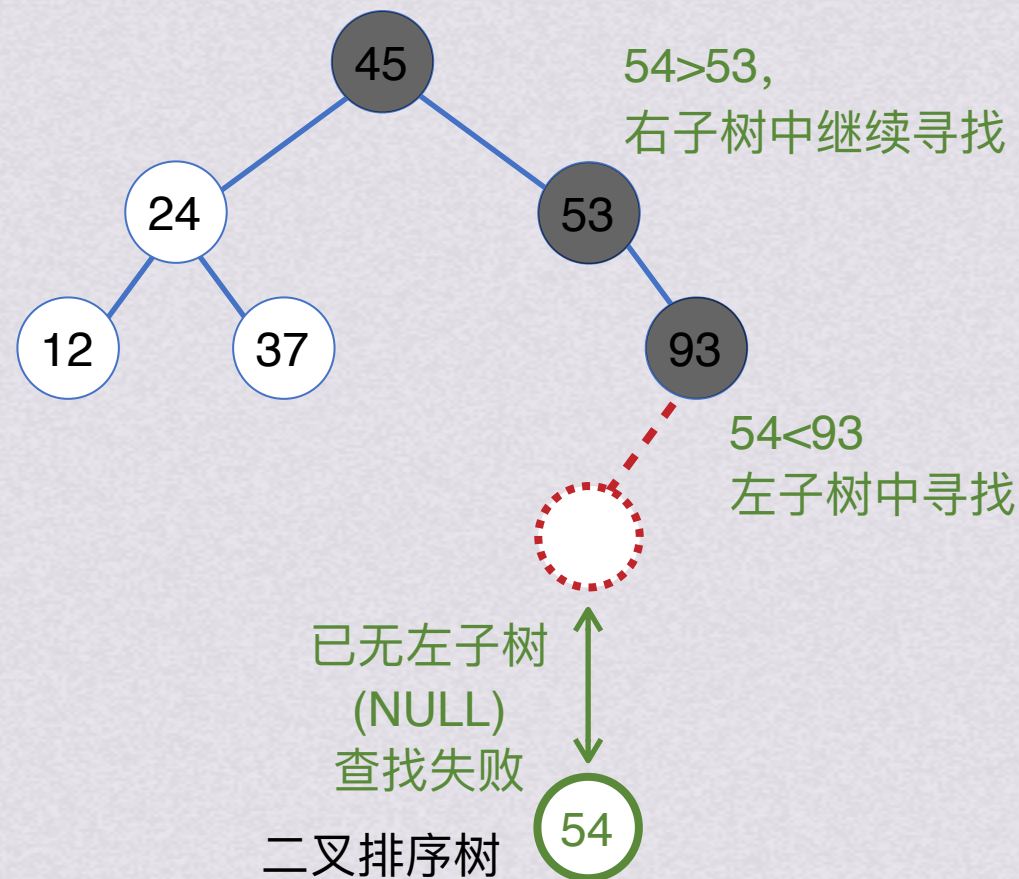
查找规则

(二叉搜索树)

若二叉排序树为空，则查找失败，返回空指针

- 若二叉排序树非空，将给定值key与根节点的关键字T->data.key进行比较
 - 若key等于T->data.key，则查找成功，返回根节点地址
 - 若key小于T->data.key，则递归查找左子树
 - 若key大于T->data.key，则递归查找右子树

54>45，右子树中继续寻找



例2：在该二叉排序树中查找key=54



二叉排序树

代码呈现

(二叉搜索树)

```
BSTree SearchBST(BSTree T,KeyType key)
{ // 在根指针 T 所指二叉排序树中递归地查找某关键字等于 key 的数据元素
  // 若查找成功，则返回指向该数据元素节点的指针，否则返回空指针
  if((!T) || key==T->data.key) return T;           //查找结束
  else if(key<T->data.key) return SearchBST(T->lchild,key); //在左子树中继续查找
  else return SearchBST(T->rchild,key);             //在右子树中继续查找
}
```



二叉排序树

ASL计算

(Average Search Length)

(二叉搜索树)

平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

$$\sum_{i=1}^n p_i c_i$$

其中， P_i 为查找表中第 i 个记录的概率，且 $\sum_{i=1}^n p_i=1$ ，一般情况下，查找表中关键字的 P_i 都默认相同，并且为 $\frac{1}{n}$

C_i 为找到表中其关键字与给定值相等的第 i 个记录所需进行的对比次数

计算出查找表中每个关键字对应的 P_i 和 C_i 相乘并求和后即为查找表查找成功的平均查找长度ASL

二叉排序树 ASL计算

(Average Search Length)

$$\sum_{i=1}^n p_i c_i$$

(二叉搜索树)

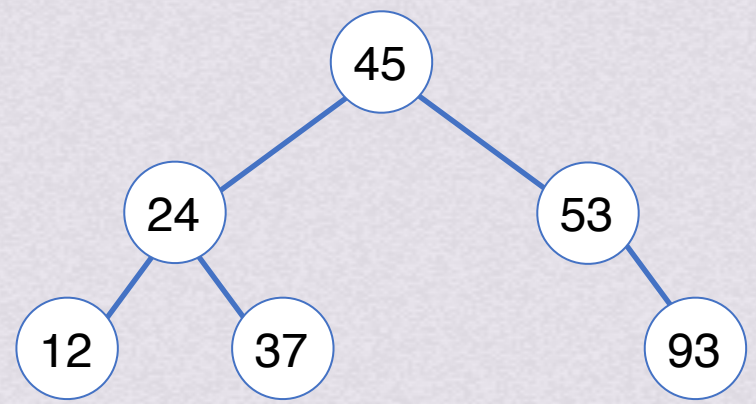
平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

其中， P_i 为查找表中第*i*个记录的概率，且 $\sum_{i=1}^n p_i=1$ ，一般情况下，查找表中关键字的 P_i 都默认相同，并且为 $\frac{1}{n}$

C_i 为找到表中其关键字与给定值相等的第*i*个记录所需进行的对比次数

计算出查找表中每个关键字对应的 P_i 和 C_i 相乘并求和后即为查找表查找成功的平均查找长度ASL

对于T，其ASL为 $ASL_T = \frac{1}{6}(1 + 2 + 2 + 3 + 3 + 3) = \frac{14}{6} = \frac{7}{3}$



二叉排序树T

二叉排序树 ASL计算

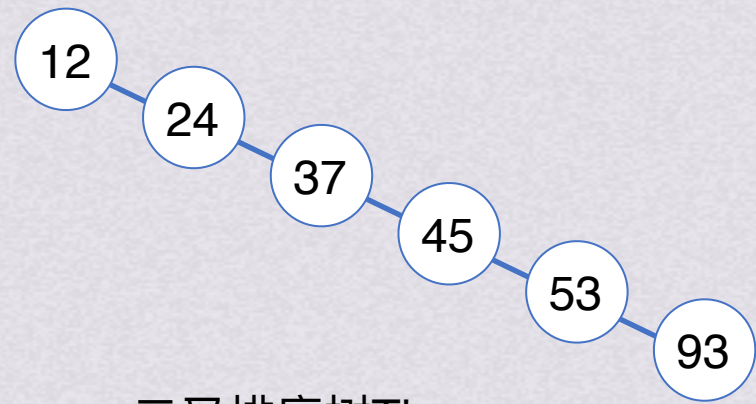
(Average Search Length)

$$\sum_{i=1}^n p_i c_i$$

(二叉搜索树)
平均查找长度ASL：为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度

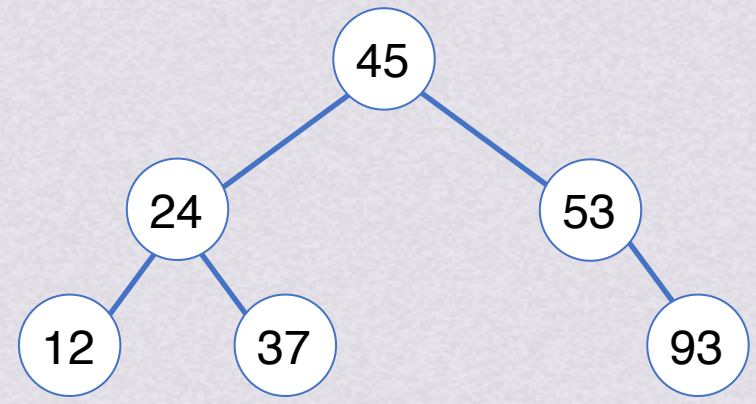
其中， P_i 为查找表中第*i*个记录的概率，且 $\sum_{i=1}^n p_i=1$ ，一般情况下，查找表中关键字的 P_i 都默认相同，并且为 $\frac{1}{n}$

C_i 为找到表中其关键字与给定值相等的第*i*个记录所需进行的对比次数
计算出查找表中每个关键字对应的 P_i 和 C_i 相乘并求和后即为查找表查找成功的平均查找长度ASL



二叉排序树T'

$$ASL_{T'} = \frac{1}{6}(1 + 2 + 3 + 4 + 5 + 6) = \frac{21}{6} = \frac{7}{2}$$



二叉排序树T

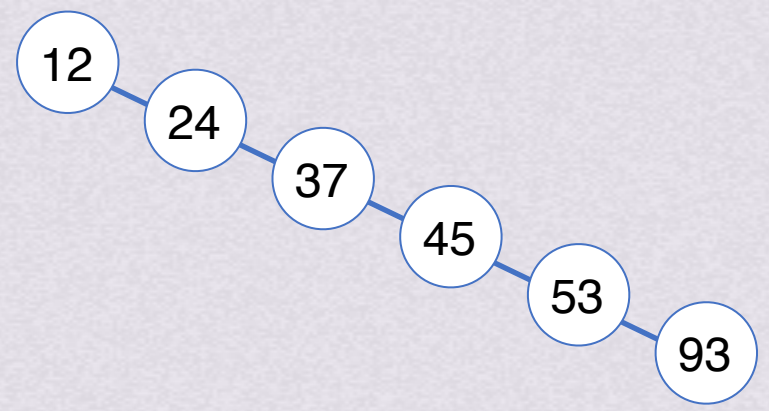
$$ASL_T = \frac{7}{3}$$

$$\sum_{i=1}^n p_i c_i$$

二叉排序树 ASL计算

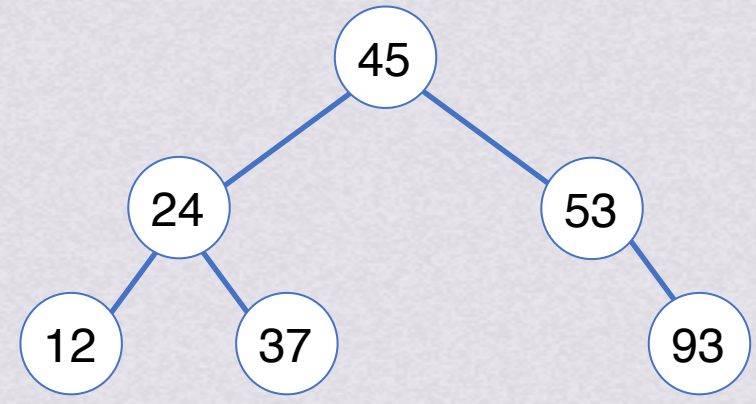
(Average Search Length)

(二叉搜索树)



二叉排序树T'

$$ASL_{T'} = \frac{7}{2}$$



二叉排序树T

$$ASL_T = \frac{7}{3}$$

二叉搜索树
(Binary Search Tree)

平衡二叉树
(英文名来自发明者)
G. M. Adelson-Velsky
E. M. Landis

对于这种树型均衡，即左右子树高度之差的绝对值不超过1的**BST**（即AVL，下节课就会学习），ASL的数量级是 $\log_2 n$

而对于这种树型不均衡的细狗树，它的ASL数量级是n

基于树是**基本平衡的平均情况下**，查找的时间复杂度为 $O(\log_2 n)$



二叉排序树

(二叉搜索树)

插入操作



二叉排序树

插入操作

(二叉搜索树)

插入操作以查找为基础，新插入的节点一定是叶子结点，
并且是查找不成功时查找路径上访问的最后一个节点的左孩子或右孩子节点

- 流程：
- 若二叉排序树为空，则将待插入节点*S作为根节点插入空树
 - 若二叉排序树非空，则将key与根节点的关键字T->data.key进行比较：
 - 若key小于T->data.key，则将*S插入左子树
 - 若key大于T->data.key，则将*S插入右子树



二叉排序树

插入操作

(二叉搜索树)

插入操作以查找为基础，新插入的节点一定是叶子结点，
并且是查找不成功时查找路径上访问的最后一个节点的左孩子或右孩子节点

流程：• 若二叉排序树为空，则将待插入节点*S作为根节点插入空树

- 若二叉排序树非空，则将key与根节点的关键字T->data.key进行比较：

- 若key小于T->data.key，则将*S插入左子树

- 若key大于T->data.key，则将*S插入右子树



二叉排序树 插入操作

(二叉搜索树)

```
// 定义一个向二叉搜索树插入新元素的函数
// T 是对二叉搜索树的引用, e 是需要插入的元素
void InsertBST(BSTree &T, ElemType e)
{
    // 如果树是空的, 或者找到了应该插入新节点的位置
    if(T == NULL)
    {
        // 创建一个新的节点
        BSTNode *S = new BSTNode;
        S->data = e; // 设置新节点的数据为e
        S->lchild = S->rchild = NULL; // 新节点的左右子节点设置为NULL
        T = S; // 将新节点链接到树上
    }
    else if(e.key < T->data.key)
        // 如果e的键小于当前节点的键, 递归地在当前节点的左子树中插入e
        InsertBST(T->lchild, e);
    else if(e.key > T->data.key)
        // 如果e的键大于当前节点的键, 递归地在当前节点的右子树中插入e
        InsertBST(T->rchild, e);
    // 如果e的键等于当前节点的键, 则不进行操作, 因为BST中不允许键值重复
}
```

二叉排序树插入的基本过程是查找, 所以时间复杂度同查找一样,
是 $O(\log_2 n)$



二叉排序树

(二叉搜索树)

删除操作



二叉排序树删除操作

(二叉搜索树)

流程：• 若删除的节点为叶子结点：

- 直接删除

• 若删除的节点不是叶子结点：

- 删除节点有左或右孩子：删除后左/右孩子上位；

- 删除节点有左、右孩子：查找删除节点右子树的最左下孩子，让其与删除节点交换，转换为删除节点为叶子结点的情况



二叉排序树

删除操作 代码部分

(二叉搜索树)

```
void DeleteBST(BSTree &T, KeyType key) {
    BSTree p = T, f = NULL; // p为当前节点, f为p的父节点
    // 从根节点开始查找关键字等于key的节点*p
    while (p) {
        if (p->data.key == key) break; // 找到目标节点, 跳出循环
        f = p; // 更新父节点
        if (p->data.key > key) p = p->lchild; // 向左子树查找
        else p = p->rchild; // 向右子树查找
    }
    if (!p) return; // 没有找到目标节点, 直接返回

    BSTree q = p;
    // 被删除节点*p左右子树均不空
    if (p->lchild && p->rchild) {
        BSTree s = p->lchild; // 寻找*p的左子树中的最大节点
        while (s->rchild) { // 寻找最右节点
            q = s; // q指向s的父节点
            s = s->rchild;
        }
    }
```

```
        p->data = s->data; // 将s的数据复制到p中
        // 调整s的父节点的子节点指针
        if (q != p) q->rchild = s->lchild; // s不是p的直接左子节点
        else q->lchild = s->lchild; // s是p的直接左子节点
        delete s; // 删除节点s
    } else if (!p->rchild) { // 无右子树, 直接用左子树替换
        p = p->lchild;
    } else if (!p->lchild) { // 无左子树, 直接用右子树替换
        p = p->rchild;
    }

    // 将p所指的子树挂接到其双亲节点f相应的位置
    if (!f) T = p; // f为NULL, 意味着删除的是根节点
    else if (q == f->lchild) f->lchild = p; // p是f的左子节点
    else f->rchild = p; // p是f的右子节点
    delete q; // 删除原节点q
}
```

同插入一样, 删除的基本操作也是查找, 因此时间复杂度仍然是 $O(\log_2 n)$

对于需要经常进行插入、删除和查找运算的表, 采用二叉排序树比较好



二叉排序树

(二叉搜索树)

创建操作



二叉排序树

创建操作

(二叉搜索树)

- 流程：
- 将二叉排序树T初始化为空树
 - 读入一个关键字为key的节点
 - 如果读入的关键字key不是输入结束标志，则循环执行以下操作：
 - 将此节点按照插入的流程插入到二叉排序树T中
 - 读入一个关键字为key的节点



二叉排序树创建操作

(二叉搜索树)

- 流程：
- 将二叉排序树T初始化为空树
 - 读入一个关键字为key的节点
 - 如果读入的关键字key不是输入结束标志，则循环执行以下操作：
 - 将此节点按照插入的流程插入到二叉排序树T中
 - 读入一个关键字为key的节点

插入序列：

45

24

53

12

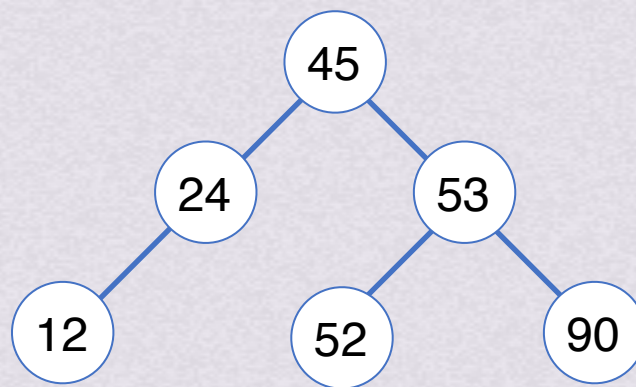
90



二叉排序树创建操作

(二叉搜索树)

- 流程：
- 将二叉排序树T初始化为空树
 - 读入一个关键字为key的节点
 - 如果读入的关键字key不是输入结束标志，则循环执行以下操作：
 - 将此节点按照插入的流程插入到二叉排序树T中
 - 读入一个关键字为key的节点



插入序列：



二叉排序树

创建操作
代码部分

(二叉搜索树)

```
void CreatBST(BSTree &T) {  
    T = NULL; // 初始化二叉搜索树，根节点设置为NULL  
    cin >> e; // 从标准输入读取一个元素到e  
    while (e.key != ENDFLAG) { // 当读取的元素的关键字不是结束标志时  
        InsertBST(T, e); // 调用InsertBST函数，将当前元素e插入到二叉搜索树T中  
        cin >> e; // 继续从标准输入读取下一个元素到e  
    }  
}
```

假设有 n 个节点，则需要 n 次插入操作，而插入一个节点的算法时间复杂度为 $O(\log_2 n)$
所以创建二叉排序树算法的时间复杂度为 **$O(n \log_2 n)$**



二叉排序树

(二叉搜索树)

查找

$O(\log_2 n)$

插入

$O(\log_2 n)$

删除

$O(\log_2 n)$

创建

$O(n) * O(\log_2 n)$
 $O(n \log_2 n)$